

5. 팀원 롤 기획서

- 프로젝트: **TeachAndFight** (Unity 6 / 2D / URP, LLM 기반 자연어 훈련 격투 게임)
- 팀 구성: 2인 팀
- 근거 문서: [docs/IMPLEMENTATION_PLAN.md](#), [docs/DEV_SPEC.md](#), GitHub 이슈 #3~#26, 커밋 히스토리

1. 팀원별 이름 · 담당 역할

이름	역할	기획·개발·아트·AI 활용 구분	GitHub 계정 / 브랜치
김진 (본인)	개발자 A — 전투 코어	기획(스키마·수치 설계) + 개발(전투 시스템) + AI 코딩 에이전트(Claude Code) 오케스트레이션	GameClientDeveloperKimJin / dev_KJ
김재민	개발자 B — AI 레이어 · UI · 아트	개발(LLM 연동·화면) + AI 생성 아트 리소스 전담 제작(캐릭터 스프라이트·애니메이션)	wls6189 / dev_JM



전체 구현은 **Claude Code(AI 코딩 에이전트)**가 수행하고, 두 사람은 각자 담당 이슈를 AI에게 지시·검토·커밋하는 역할을 맡았다. 즉 "누가 어떤 코드를 타이핑 했는가"보다 "누가 어떤 영역의 설계·검증·의사결정을 책임졌는가"로 역할이 나뉜다.

2. 역할 분담 기준 — 점점(스키마)으로 A/B를 가른다

[docs/DEV_SPEC.md](#) 01장 규칙 스키마를 공용 계약으로 두고, 그 스키마를 만드는 쪽(A)과 소비하는 쪽(B)으로 작업을 쪼갬다.

	개발자 A (김진) — 전투 코어	개발자 B (김재민) — AI 레이어
담당 씬	Match.unity	Training.unity, LockerRoom.unity
담당 코드	Scripts/Combat, Scripts/Core (규칙 스키마·RuleValidator·RuleEvaluator·FighterController FSM)	Scripts/Training, UI (LLMClient·프롬프트·훈련 컴파일 파이프라인)
담당 데이터	combat_config.json, 상대 5종 규칙셋 JSON	LLM 시스템 프롬프트, 제자 대사 톤

- 규칙 스키마(01장)를 바꾸고 싶으면 **먼저 상대에게 합의를 구하고 version을 올린 뒤 양 쪽 코드를 동시 반영**하는 걸 원칙으로 삼았다

→ 2인 팀에서 접점이 하나뿐이라, 이 지점만 지키면 나머지는 완전 병렬로 작업 가능했다.

3. 주차별(W1~W4) 역할 수행

- 4주 일정으로 쪼개고, 매주 **A/B가 서로 안 막히는 순서**로 배치해 병렬성을 최대화했다. 실제 요일이 아니라 작업 순서(Day 1~20) 기준.

주차	목표	김진(A) 담당 이슈
W1	스키마 확정 + 전투 코어 기반	#4 규칙 스키마+RuleValidator, #5 FighterController FSM, #6 하드코딩 1v1 데모
W2	규칙 실행 + LLM 파이프라인 연동	#7 RuleEvaluator(0.1s 틱 의사결정), #8 EventLog
W3	콘텐츠 + 화면 3종	#12 상대 5종 규칙셋(러쉬/철벽/그림자/카멜레온/사범), #13 Match.unity
	공용	#16 GameFlow 씬 전환(A/B 공동), #22 캐릭터 비주얼 에셋 확보(A/B 공동 소싱) — Animator 적용 코드는 워크로드 격차 보정 위해 B 단독
W4	폴리싱 + 밸런싱 + 데모	#17 연출 폴리싱(슬로모·배속), #18 밸런싱 플레이테스트, 추가로 needs_confirmation UX 개선
	공용	#20 데모 빌드 + 최종 QA(00~05장 전체 완료 기준 재확인)



#16(GameFlow)처럼 A/B 접점이 되는 이슈는 시작 전에 인터페이스 (GameSession/MatchResult 클래스 시그니처)부터 합의하고 나서 각자 구현했다. 이 계약을 Day 1에 가장 먼저 고정해둔 덕분에 W3에서 화면 3종을 완전 병렬로 만들 수 있었다.

4. 주차 전환 시 AI와 함께 검증·테스트하는 방식 (마일스톤 게이트)

- 이번 프로젝트의 핵심 협업 규칙
⇒ "Wn의 이슈가 개발자A + 개발자B + 공용 구분 없이 전부 closed일 때만 Wn+1을 시작한다." 이걸 사람이 매번 기억해서 챙기는 대신, AI 세션 시작 프로토콜로 강제했다.

검증 절차 (매 세션 자동 반복)

1. 상태 조회

- Claude Code가 세션을 시작할 때마다 `gh issue list --state all`, `git log --all`, `git branch -a`로 팀원의 깃허브 이슈 및 커밋 현황을 스스로 조회한다.

```
(base) PS D:\UnityProject\2D\TeachAndFight> claude
Claude Code v2.1.223
Sonnet 5 - Claude Pro
D:\UnityProject\2D\TeachAndFight

[SessionStart:startup says: [세션 시작 안내] '/gogo' 입력해서 현재 w단계 진행 상태 + 팀원 브랜치 상태 확인할 것]
```

2. Wn 판정 (주차별 작업 여부 판정)

- 해당 주차에 속한 Github 이슈(A 담당 + B 담당 + 공용)가 전부 closed인지 확인하고, 사용자에게 먼저 보고한다.

☐	Open	0	Closed	23	Author
☐	☒				
☐	☒				
☐	☒				
☐	☒				
☐	☒				

3. 하드 게이트

- 하나라도 Github 이슈가 open이라면(내 담당이 아니라 팀원 담당이라도) 다음 주 차 코드 작성을 거부한다.
→ "누구의 어떤 이슈가 막고 있는지"를 이유와 함께 알려주고 작업을 중단한다.

4. 완료 기준(✓) 재확인

- 이슈가 closed라는 표시만으로 끝내지 않고, 각 이슈에 적힌 완료 기준(예: "RuleValidator 단위테스트 3종 통과", "백지 규칙셋으로 1차전 반드시 패배")을 실제로 통과했는지까지 의심되면 다시 열어 확인한다.



예시: W2에서 B의 #10/#11이 아직 open인 상태로 "W3 시작하자"고 요청해도, AI는 A(김진) 담당 여부와 무관하게 진입을 거부하고 왜 막혔는지(이슈 번호·담당자·근거 장) 를 먼저 알려주도록 규칙을 세웠다.

- 이 방식의 효과: 2인 팀이라 서로 진행 상황을 매번 구두로 맞춰볼 여유가 적는데, AI가 이슈/브랜치 상태를 스스로 조회해서 게이트를 지켜주니 "한쪽만 끝난 줄 알고 다음 단계로 넘어갔다"가 나중에 배관이 안 맞는" 사고를 원천 차단할 수 있었다.
⇒ 실제로 W3 → W4 전환 때 #16 GameFlow 순환 검증을 A/B 공동으로 먼저 확인한 뒤에야 폴리싱 단계로 넘어갔다.

5. 협업·분업 방식 — GitFlow

실제 브랜치 구조

```
main (항상 데모 가능한 안정 상태)
├─ develop (양쪽 작업이 합쳐지는 통합 브랜치)
│   ├─ dev_KJ (김진 개인 작업 브랜치)
│   └─ dev_JM (김재민 개인 작업 브랜치)
```

Default branch

main

Recent branches

✓ dev_KJ

develop

dev_JM

- 각자 `dev_{이름}` 브랜치에서 자유롭게 커밋하다가, 이슈 완료 기준을 통과하면 **Pull Request로 develop에 병합**(예: `Merge pull request #21/#23/#25 from ../dev_KJ`).
- develop에서 안정성이 확인된 시점에만 main으로 다시 병합한다.
- 커밋 메시지는 `[영역] 내용` 형식으로 태그 접두(`combat / training / core / match` 등)를 붙여, 로그만 봐도 어느 담당 영역 작업인지 바로 구분되게 했다.

이 방식으로 얻은 이점

- **개인 브랜치 자유도 + 통합 브랜치 안정성 동시 확보**: dev_KJ/dev_JM에서는 실험적인 커밋을 마음껏 남겨도 develop이 오염되지 않았고, PR 병합 시점에만 서로 코드를 확인하면 됐다.
- **PR이 자연스러운 리뷰 지점이 됨**: 씬/프리팹처럼 충돌 위험이 큰 파일은 "PR 병합 전 상호 확인"을 규칙으로 못박아, 별도 리뷰 프로세스 없이도 사고를 예방했다.
- **main은 항상 제출 가능한 상태 유지**: 실험 중인 브랜치가 아무리 지저분해도 main은 항상 "지금 당장 빌드해도 되는" 상태였다 — 데모 제출 직전 결함 발견 리스크를 크게 줄임.
- **2인 팀 규모에 맞춘 경량화**: 정식 GitFlow(feature/release/hotfix 브랜치 세분화)를 그대로 다 쓰지 않고, `main-develop-개인브랜치` 3단만 남겨 관리 부담을 최소화하면서도 "안정 브랜치/작업 브랜치 분리"라는 핵심 이점은 그대로 가져갔다.

- 장기 브랜치 금지 원칙: "2인 팀에서 3일 묵은 브랜치는 사고"라는 기준을 세우고 매일 develop에 병합하는 걸 원칙으로 삼아, 병합 충돌이 쌓이는 걸 방지했다.

6. 각 팀원이 실제로 맡아 구현한 영역

김진 (개발자 A)

- 01장 규칙 스키마 C# 모델 + `RuleValidator` (#4)
- `FighterController` FSM + 스탯/스킬 4종, `combat_config.json` 수치 체계 (#5)
- `RuleEvaluator` (0.1초 틱 의사결정 루프) + `EventLog` 시스템 (#7, #8)
- 상대 5종 규칙셋 JSON(러쉬/철벽/그림자/카멜레온/사범) 밸런스 설계 (#12)
- `Match.unity` 화면(HUD·타이머·규칙 발동 라벨·배속) (#13)
- needs_confirmation 되묻기 UX 개선 — 플레이테스트 중 직접 발견해 담당 밖 작업을 자원해서 추가 처리
- 세션 후반: 어휘 확장(counter_attack/feint 등 신규 액션·fact), 밸런스 패치, 다수 QA 버그 수정(교착 상태 감지, 승리 후 다음 상대 진행 로직 등)

김재민 (개발자 B)

- `ILLMClient` / `AnthropicLLMClient` (Claude Haiku 연동) + 03장 시스템 프롬프트 빌더 (#9)
- 훈련 컴파일 파이프라인(`TrainingCompiler` + `RuleDiff` 파서) (#10)
- 프롬프트 인젝션 방어 테스트 3종 (#11)
- `Training.unity` 화면(규칙 슬롯·대화 로그·가르치기 UI) (#14)
- `LockerRoom.unity` 화면 + 경기 회고 LLM 연동 (#15)
- **AI 생성 캐릭터 아트 리소스 전 과정 담당** — 스프라이트·애니메이션 클립 제작, `FighterAnimatorBridge` 연동, 프리뷰 씬 구성 (#22, `docs/CHARACTER_ART_PLAN.md` 기준 6 캐릭터 × 8클립 규모)



캐릭터 비주얼(#22)은 원래 이슈 수 워크로드가 A(9개) > B(6개)로 불균형해서, **에셋 소싱은 공동으로 하되 AI 이미지 생성부터 스프라이트 슬라이스·Animator 연동까지 전체 아트 파이프라인을 김재민이 단독으로 전담하는 것으로 조율했다.** 물량이 예상보다 커져(48클립) W3 → W4 마일스톤 게이트에서는 제외하고 별도 병렬 트랙으로 진행했다.